Univerzitet u Sarajevu
Elektrotehnički Fakultet

Objektno Orijentisana Analiza i Dizajn

ETF Talent Program

Task 5 — Dijagram klasa i SOLID principi

Grupa 1, Tim 2

April 2026.

Sadržaj

1 Dijagram klasa **2**
1.1 Domenski model — korisnici i profili 2
1.2 Transakcijske klase 2

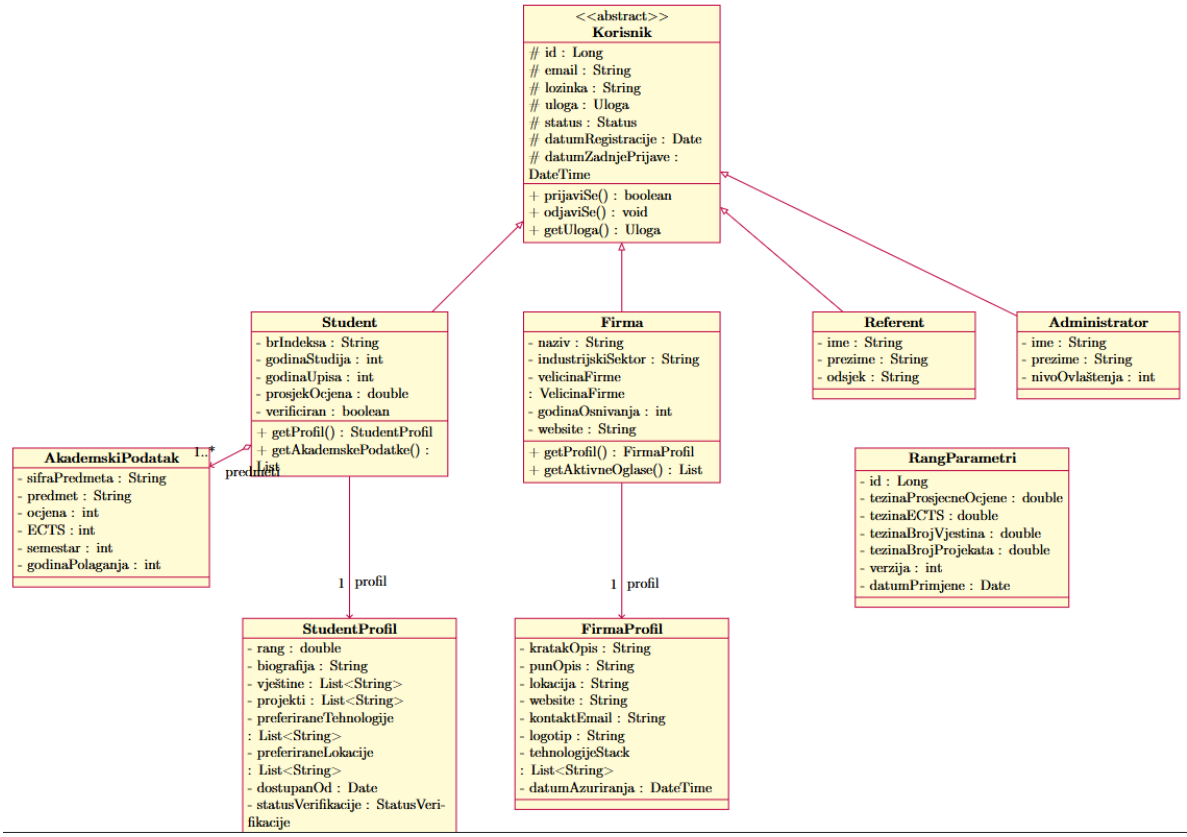
2 SOLID principi u sistemu ETF Talent Program **4**

1. Dijagram klasa

Dijagram klasa prikazuje statičku strukturu sistema ETF Talent Program. Klase su organizovane u dva logička dijela prema domeni kojoj pripadaju.

1.1. Domenski model — korisnici i profili

Ovaj dio dijagrama obuhvata hijerarhiju korisničkih klasa (`Korisnik`, `Student`, `Firma`, `Referent`, `Administrator`) zajedno s profilnim klasama (`StudentProfil`, `FirmaProfil`) i pratećim klasama `AkademskiPodatak` i `RangParametri`.

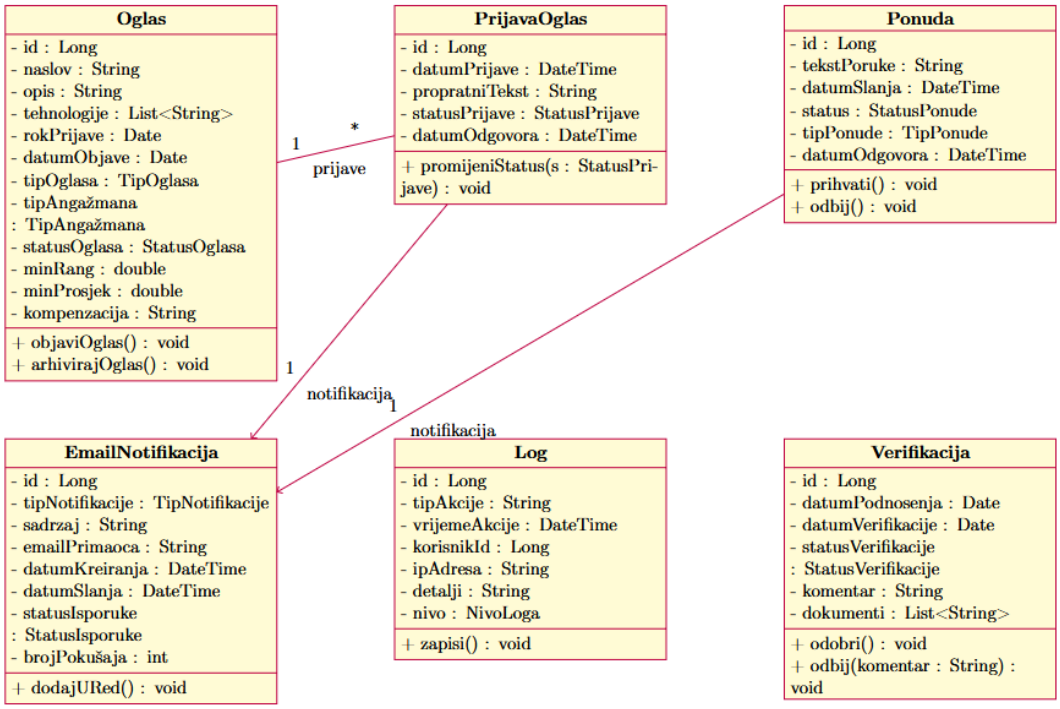


Slika 1: Domenski model — korisnici, profili i akademski podaci

Klasa `Korisnik` je apstraktna i sadrži zajedničke atribute (email, lozinka, uloga, status) te metode za prijavu/odjavu. Nasljeđuju je četiri konkretne klase: `Student`, `Firma`, `Referent` i `Administrator`. Svaki `Student` posjeduje tačno jedan `StudentProfil` koji čuva rang, vještine i preferencije, dok svaka `Firma` posjeduje tačno jedan `FirmaProfil`. Rang studenta izračunava se algoritmom čiji su ponderi definirani u klasi `RangParametri`.

1.2. Transakcijske klase

Ovaj dio dijagrama modelira poslovne radnje sistema: objavljivanje oglasa, prijave studenata, slanje ponuda, email obavijesti, logovanje i verifikaciju.



Slika 2: Transakcijske klase — Oglas, PrijavaOglas, Ponuda, EmailNotifikacija, Log, Verifikacija

Klasa `Oglas` sadrži sve detalje radne pozicije ili prakse. Na jedan oglas može biti vezano više instanci `PrijavaOglas` (veza $1 : *$). Slanjem prijave ili ponude automatski se kreira `EmailNotifikacija` koja se isporučuje putem eksternog servisa. Klasa `Log` bilježi svaku ključnu akciju u sistemu, dok `Verifikacija` prati tok provjere akademskih podataka od strane Referenta.

2. SOLID principi u sistemu ETF Talent Program

U nastavku su opisani svi pet SOLID principa objektno orijentisanog dizajna i konkretni primjeri njihove primjene u ovom projektu.

S Single Responsibility Principle (SRP)

Svaka klasa treba imati tačno jednu odgovornost, tj. jedan razlog za promjenu.

- **Korisnik** — isključivo autentifikacija i životni ciklus naloga (`prijaviSe()`, `odjaviSe()`, status); ne zna ništa o poslovnoj logici oglasa ili ponuda.
- **StudentProfil** — upravljanje profilnim podacima studenta (rang, vještine, preferencije); odvojen od klase `Student` koja čuva samo identifikacijske podatke.
- **RangParametri** — jedino zadužen za konfiguraciju algoritma rangiranja (ponderi, verzija); promjena formule ne zahtijeva izmjenu klase `StudentProfil`.
- **EmailNotifikacija** — jedino zadužena za slanje i praćenje email poruka (`dodajURed()`); ne poznaje poslovnu logiku ponuda ni prijava.
- **Log** — jedino loguje akcije u sistemu (`zapisi()`); Administrator dobija uvid u rad sistema bez poznavanja detalja poslovnih objekata.

O Open/Closed Principle (OCP)

Klase trebaju biti otvorene za proširenje, ali zatvorene za izmjenu.

- Apstraktna klasa **Korisnik** definiše stabilan interfejs; dodavanje novog tipa korisnika (npr. `Mentor`) zahtijeva samo kreiranje nove podklase — bez izmjene postojećeg koda.
- Atribut `verzija` u klasi **RangParametri** omogućava uvođenje novih verzija algoritma rangiranja bez mijenjanja klase `StudentProfil`.
- Enumaracije `TipOglasa`, `TipAngažmana` i `StatusOglasa` u klasi **Oglas** omogućavaju dodavanje novih vrijednosti (npr. novi tip angažmana) bez promjene poslovne logike.
- Metode **Ponude** (`prihvati()`, `odbij()`) i **Verifikacije** (`odobri()`, `odbij(komentar)`) definišu stabilan interfejs koji se može proširiti bez mijenjanja klijentskog koda.

L Liskov Substitution Principle (LSP)

Objekti podklase moraju moći zamijeniti objekte nadklase bez narušavanja korektnosti programa.

- **Student**, **Firma**, **Referent** i **Administrator** nasljeđuju `Korisnik` — svaka konkretna klasa može se proslijediti tamo gdje se očekuje `Korisnik` (npr. pri prijavi, logovanju, provjeri statusa).
- Klasa **Log** referencira `korisnikId : Long` koji važi za bilo koji tip korisnika; promjena konkretnog tipa aktera ne narušava mehanizam logovanja.
- Metode `prijaviSe()` i `odjaviSe()` definirane su u `Korisnik` i ispravno funkcionišu za sve podklase bez potrebe za posebnom obradom po tipu.

I Interface Segregation Principle (ISP)

Klijenti ne smiju biti primorani da ovise o metodama koje ne koriste.

- **StudentProfil** izlaže atribute relevantne samo za studenta (rang, vještine, akademski status), dok **FirmaProfil** izlaže poslovne podatke firme — firma nikad ne vidi rang-specifične atribute, a student ne vidi poslovne podatke firme.
- **Referent** koristi isključivo operacije `Verifikacija` i upravljanje korisnicima; ne dolazi u kontakt s klasama `Oglas` ili `Ponuda`.
- **Administrator** radi isključivo s klasom `Log` (`getUloga()`, pregled logova) i nema pristup poslovnim operacijama.
- **Oglas** izlaže samo `objaviOglas()` i `arhivirajOglas()` — Firma ne mora znati ništa o internoj implementaciji logike prijava ili notifikacija.

D Dependency Inversion Principle (DIP)

Moduli visokog nivoa ne smiju ovisiti o modulima niskog nivoa; oba trebaju ovisiti o apstrakcijama.

- **EmailNotifikacija** komunicira s eksternim email servisom kroz apstraktni interfejs — konkretna implementacija (SendGrid, SMTP, ...) može se zamijeniti bez izmjene klase `EmailNotifikacija`.
- Algoritam rangiranja u **StudentProfil** ovisi o apstrakciji **RangParametri**, a ne o konkretnim vrijednostima pondisanja — ponderi se mogu promijeniti (nova verzija) bez izmjene profilne logike.
- **Verifikacija** referenciše `Korisnik` (apstrakciju), a ne direktno klasu `Student`, čime se olakšava buduća proširivost procesa verifikacije na druge tipove korisnika.
- **Log** referenciše `korisnikId : Long` umjesto direktne reference na konkretnu korisničku klasu, što znači da logika logovanja ne ovisi ni o jednoj konkretnoj podklasi korisnika.